CESDE

Santiago Yosa González

Ingeniero Sistemas de Información

CESDE | comfama

Unidad I

Fundamentación técnica en Java (estructuras y modularidad)

- **Colecciones en Java:** List (ArrayList) y operaciones básicas
- **Estructuras clave–valor:** Map (HashMap) y casos de uso
- **Modularidad y reutilización:** métodos (funciones), parámetros, retorno

Fundamentos de POO

Situación Inicial: El Restaurante Gourmet

Imagina que entras a la cocina de un restaurante.

No hay una sola persona haciendo todo; hay una estación de salsas, una de carnes y una de postres. Cada estación tiene **procesos (métodos)** que reciben ingredientes **(parámetros)** y entregan un plato **(retorno)**. Si la estación de carnes cambia su parrilla, la de postres ni se entera.

Eso es **Modularidad**.

En Java, no escribimos un "supercódigo" infinito, construimos piezas pequeñas y especializadas.

Colecciones en Java

Son una estructura de datos dentro del paquete java.util diseñada para almacenar, manipular y agrupar objetos como una sola unidad. Proporcionan una alternativa dinámica y más flexible a los arrays, permitiendo añadir, eliminar, ordenar y buscar elementos eficientemente. Las principales interfaces son List (ordenada, admite duplicados), Set (elementos únicos) y Map (pares clave-valor).

Beneficios de colecciones en Java

Tiene los siguientes beneficios.

1. Ya implementado (ahorro de tiempo).
2. Rendimiento Eficiencia (rapidez y calidad).
3. Reduce el esfuerzo para aprender y utilizar nuevas API.

Colecciones en Java - List (ArrayList)

¿Por qué no usar Arrays simples?

En lógica de programación tradicional, los arreglos tienen un tamaño fijo definido en tiempo de compilación. En Java, si declaras int[] numeros = new int[5]; no puedes agregar un sexto elemento. El **ArrayList** es una implementación de la interfaz List que utiliza un arreglo interno que se redimensiona automáticamente (crece un **50%** de su capacidad cada vez que se llena).

Teoría Completa y Conceptos Clave

1. **Tipos Genéricos (Generics):** Java utiliza <T> para definir qué tipo de objeto guardará la lista. Esto garantiza seguridad de tipos.
2. **Wrapper Classes (Clases Envolventes):** Las colecciones NO pueden guardar primitivos (int, double, char). Deben usar sus versiones de objeto: Integer, Double, Character.
3. **Índices:** Al igual que en la lógica básica, el primer elemento es el 0.
4. **Memoria:** Un ArrayList mantiene el orden de inserción y permite elementos duplicados

Tipos primitivos en Java

Son los tipos más básicos, no son objetos y ocupan poca memoria.

Tipo	Tamaño	Valor por defecto	Ejemplo	Uso frecuente
byte	8 bit	0	byte edad = 18;	Datos pequeños
short	16 bit	0	short año = 2024;	Enteros medianos
int	32 bit	0	int edad = 25;	Enteros generales
long	64 bit	0L	long poblacion = 8000000000L;	Números muy grandes
float	32 bit	0.0f	float nota = 4.5f;	Decimales simples
double	64 bit	0.0	double precio = 4999.99;	Decimales precisos
char	16 bit	'\u0000'	char letra = 'A';	Caracteres
boolean	1 bit	false	boolean activo = true;	Verdadero/Falso

Clases Envolventes (Wrapper Class)

Una *clase envolvente* (wrapper class) **"envuelve"** un tipo de dato primitivo dentro de un objeto, lo que permite que pueda usarse en contextos donde se requieren objetos.
Cada tipo primitivo tiene una clase equivalente que permite **tratarlo como objeto**.

Primitivo	Clase envolvente
int	Integer
double	Double
boolean	Boolean
char	Character

¿Para qué sirven?

Guardar datos en estructuras como ArrayList
Trabajar con valores null
Usar métodos útiles como parseInt(), toString(), compareTo(), etc.
Convertir valores primitivos a objetos
Convertir Strings a números

Autoboxing y Unboxing, aplican de primitivo a objeto y objeto a primitivo, respectivamente.

Mientras que Casteo conversión de un tipo de dato a otro tipo de dato compatible (double, int, long).

Finalmente, el parseo de texto a otro tipo de dato usando wrappers.

Autoboxing: Java convierte automáticamente de primitivo a objeto.
Unboxing: Java convierte automáticamente de objeto a primitivo.

```
int numero = 5;                  // tipo primitivo
Integer numeroObjeto = numero;   // autoboxing: se convierte automáticamente
int otroNumero = numeroObjeto;   // unboxing: de objeto a primitivo
```

Casteo (casting)
Convertir entre tipos compatibles

- **Implícito:**
  ```
  int a = 10;
  double b = a;
  ```

- **Explícito:**
  ```
  double x = 5.7;
  int y = (int) x;  // y = 5
  ```

Parseo:

```
int numero = Integer.parseInt(texto); // Parseo: "456" (String) -> 456 (int)

String edadComoTexto = Integer.toString(edad); // "30"
```

Anatomía y Operaciones Básicas - List (ArrayList)

- **add(E e):** Añade al final.
- **add(int index, E e):** Inserta en una posición desplazando el resto.
- **get(int index):** Recupera sin borrar.
- **set(int index, E e):** Reemplaza el valor.
- **remove(int index) o remove(Object o):** Elimina el elemento.

Demo Paso a Paso: Gestión de Inventario Físico

```
import java.util.ArrayList; // Necesario para usar la clase
import java.util.List;      // Recomendado instanciar desde la interfaz

public class GestionInventario {
    public static void main(String[] args) {
        // 1. Declaración e Instanciación
        // Usamos List (interfaz) a la izquierda y ArrayList a la derecha
        List<String> productos = new ArrayList<>();

        // 2. Operación: Escritura (Create)
        productos.add("Laptop");
        productos.add("Mouse");
        productos.add("Teclado");

        // 3. Operación: Lectura (Read)
        System.out.println("Primer producto: " + productos.get(0));

        // 4. Operación: Actualización (Update)
        // Cambiamos 'Mouse' por 'Mouse Inalámbrico' en el índice 1
        productos.set(1, "Mouse Inalámbrico");

        // 5. Operación: Eliminación (Delete)
        productos.remove("Teclado"); // Por objeto

        // 6. Recorrido (Iteración)
        System.out.println("--- Inventario Actual ---");
        for (String p : productos) {
            System.out.println("Producto: " + p);
        }

        // Pormenor: Tamaño de la lista
        System.out.println("Total productos: " + productos.size());
    }
}
```

Estructuras Clave-Valor - Map (HashMap)

A diferencia de una lista donde para buscar un dato podrías tener que recorrerla toda (O(n)), un HashMap usa una función *Hash* para calcular la posición exacta. Es ideal para cuando tienes un identificador único (ID, Cédula, SKU) y quieres obtener un registro sin iterar.

Conceptos Clave

1. **Par K(clave), V(valor) (Key, Value):** La **Key** (llave) debe ser única; si intentas insertar una llave existente, el valor se sobrescribe. El **Value** puede repetirse.
2. **Nullability:** HashMap permite una llave nula y múltiples valores nulos.
3. **No hay orden:** A diferencia del ArrayList, el HashMap no garantiza que los elementos salgan en el mismo orden en que los pusiste.
4. **Uso de memoria:** Consume más memoria que una lista porque guarda la relación y el valor del hash.
5. **Sobrescritura**: Si se inserta una llave que ya existe, el valor anterior se pierde y se reemplaza por el nuevo.

Casos de Uso

- Diccionarios de datos.
- Caché de información de usuarios por ID.
- Conteo de frecuencias (ej. cuántas veces aparece una palabra).

Demo Paso a Paso: Registro de Calificaciones

```
import java.util.HashMap;
import java.util.Map;

public class SistemaNotas {
   public static void main(String[] args) {
      // 1. Instanciación: Llave (String - Nombre), Valor (Double - Nota)
      Map<String, Double> notasEstudiantes = new HashMap<>();

      // 2. Insertar datos (put)
      notasEstudiantes.put("Santiago", 4.5);
      notasEstudiantes.put("Laura", 5.0);
      notasEstudiantes.put("Carlos", 3.8);

      // 3. Recuperar un valor (get)
      Double notaLaura = notasEstudiantes.get("Laura");
      System.out.println("Nota de Laura: " + notaLaura);

      // 4. Verificar existencia
      if (notasEstudiantes.containsKey("Carlos")) {
         System.out.println("Carlos está registrado.");
      }

      // 5. Borrado
      notasEstudiantes.remove("Carlos");

      // 6. Recorrido de llaves y valores (EntrySet)
      System.out.println("--- Listado de Notas ---");
      for (Map.Entry<String, Double> entrada : notasEstudiantes.entrySet()) {
         System.out.println("Estudiante: " + entrada.getKey() + " | Nota: " + entrada.getValue());
      }
   }
}
```

Modularidad - Métodos, Parámetros y Retorno

El Principio de Responsabilidad Única

Un método debe hacer **una sola cosa** y hacerla bien. Si tu método procesarPago() también imprime la factura y envía un correo, está mal diseñado. La modularidad permite que el código sea testeable y reutilizable.

Teoría Completa y Conceptos Clave

1. **Firma del Método:** Se compone del nombre y la lista de parámetros. Java permite la **Sobrecarga** (mismo nombre, diferentes parámetros).
2. **Paso por Valor:** En Java, todo se pasa por valor. Para primitivos, se pasa una copia del dato. Para objetos, se pasa una copia de la **referencia** (dirección de memoria).
3. **Tipo de Retorno (return):** Define el final del método. Si es void, no devuelve nada, solo ejecuta acciones. Si tiene un tipo (int, List, etc.), debe usar la palabra reservada return.
4. **Modificadores de Acceso:** public, private, protected. (En este nivel inicial, nos enfocamos en public para visibilidad total).

```
public static List<Integer> filtrarPares(List<Integer> entrada)
{
// Lógica interna: Entrada -> Proceso -> Salida
}
```

Demo Paso a Paso: Procesador de Impuestos

```java
import java.util.ArrayList;
import java.util.List;

public class ContabilidadApp {
    public static void main(String[] args) {
        double salarioBruto = 3500000;

        // Llamado a un método que retorna un valor simple
        double impuesto = calcularIVA(salarioBruto, 0.19);
        System.out.println("Impuesto a pagar: " + impuesto);

        // Llamado a un método que procesa colecciones
        List<String> errores = new ArrayList<>();
        validarSalario(salarioBruto, errores);

        System.out.println("Errores encontrados: " + errores);
    }

    /**
     * Calcula el valor de un impuesto dado un monto y una tasa.
     * @param monto El valor base.
     * @param tasa Porcentaje en decimal (ej: 0.19).
     * @return El valor del impuesto calculado.
     */
    public static double calcularIVA(double monto, double tasa) {
        return monto * tasa;
    }

    /**
     * Valida si un salario es legal y añade errores a una lista.
     * @param salario Monto a validar.
     * @param listaErrores Lista que se modifica por referencia.
     */
    public static void validarSalario(double salario, List<String> listaErrores) {
        if (salario < 1300000) {
            listaErrores.add("El salario es menor al mínimo legal.");
        }
    }
}
```

Null Safety (El Cinturón de Seguridad)

Antes de tocar cualquier colección, debemos entender que Java no perdona el acceso a referencias vacías.

En Java, una variable de tipo objeto que no ha sido inicializada o que apunta a "nada" tiene el valor null. Intentar acceder a un método o propiedad de algo que es null provocará que el programa "explote" (se detenga abruptamente).

¿Qué es el NullPointerException (NPE)?

Es un error de tiempo de ejecución que ocurre cuando la Máquina Virtual de Java (JVM) intenta realizar una operación sobre una referencia que no apunta a ningún objeto en memoria. Es como intentar abrir una puerta con una llave que no existe.

El peligro en las Colecciones

Las colecciones son "puntos calientes" para este error por dos razones principales:

1. **En ArrayList:** Si intentas obtener un objeto de una lista que contiene un null y luego llamas a un método sobre él.
2. **En HashMap:** Si pides una **llave que no existe**, el método .get(key) te devolverá null. Si intentas usar ese resultado sin validar, el programa fallará.

Teoría de Seguridad

- **Definición de null**: Representa la ausencia de valor en una referencia de objeto.
- **El NPE (NullPointerException)**: Es el error que ocurre al intentar usar un objeto que es null.
- **Estrategia de validación**: Siempre debemos preguntar if (objeto != null) antes de invocar métodos.

Antes de operar con cualquier objeto que provenga de una fuente externa o una colección, debemos aplicar un filtro de seguridad.

```
Map<String, Double> precios = new HashMap<>();
// ... supongamos que el mapa está vacío

Double precio = precios.get("Laptop"); // Esto devuelve null porque no existe

// ERROR SEGURO:
// double total = precio * 1.19; // Esto lanza NullPointerException

// FORMA SEGURA:
if (precio != null) {
   double total = precio * 1.19;
   System.out.println("El total es: " + total);
} else {
   System.out.println("Error: El producto no existe en el catálogo.");
}
```

Técnicas Modernas de Prevención

Método getOrDefault: En lugar de recibir un null, puedes pedirle al mapa que te devuelva un valor base si la llave no existe.

```
Double precio = precios.getOrDefault("Laptop", 0.0); // Devuelve 0.0 en lugar de null
```

Uso de containsKey: Verificar la existencia antes de extraer.
Clase Optional (Nivel Avanzado): Un contenedor que indica explícitamente si un valor está presente o no.

¿Por qué Java permite el valor null si es tan peligroso? Respuesta: null es útil para representar la **ausencia de un valor**.
El problema no es el null en sí, sino el programador que olvida validar su existencia antes de usarlo.
¿Es lo mismo una lista vacía que una lista null? Respuesta:
No. Una lista vacía es un objeto real (una caja vacía). Una lista null es la ausencia total del objeto (ni siquiera tienes la caja).

Casos de Uso y Demos - ArrayList: El historial dinámico

Registro de logs de actividad o carritos de compras. Se usa cuando necesitas mantener el orden en que ocurrieron los eventos y no sabes cuántos habrá.

```
import java.util.ArrayList;
import java.util.List;

public class LogSistema {
    public static void main(String[] args) {
        // Instanciación profesional usando la Interfaz List
        List<String> eventos = new ArrayList<>();

        // Operaciones básicas
        eventos.add("Usuario logueado");
        eventos.add("Error en BD");

        // Acceso con validación de tamaño
        if (!eventos.isEmpty()) {
            System.out.println("Último evento: " + eventos.get(eventos.size() - 1));
        }
    }
}
```

HashMap: El buscador ultra-rápido

Gestión de sesiones de usuario o diccionarios de configuración. Se usa cuando tienes un ID único y quieres los datos asociados *ya*, sin recorrer listas.

```
import java.util.HashMap;
import java.util.Map;

public class ConfigApp {
    public static void main(String[] args) {
        Map<String, String> ajustes = new HashMap<>();
        ajustes.put("idioma", "ES");
        ajustes.put("timeout", "30s");

        // USO DE NULL SAFETY AL RECUPERAR
        String valor = ajustes.get("tema"); // Devuelve null porque no existe

        if (valor != null) {
            System.out.println("El tema es: " + valor.toLowerCase());
        } else {
            System.out.println("Configuración 'tema' no encontrada. Usando modo claro.");
        }
    }
}
```

Modularidad: El motor de lógica

Procesamiento de reglas de negocio (impuestos, validaciones, conversiones).

```
public static double convertirCelsiusAFahrenheit(double celsius) {
return (celsius * 9/5) + 32;
}
```

Práctica Acompañada: Construyendo un "Sistema de Gestión de Talento"

Práctica 1: Ejemplo de Aprendizaje

Escenario: Vamos a crear un programa que gestione una lista de empleados y sus salarios.

Paso a paso:

1. **Estructura Base:** Crea una clase llamada GestionTalento.
2. **El Almacén:** Crea un ArrayList para los nombres de los empleados y un HashMap para asociar nombres con salarios.
3. **El Módulo de Registro:** Crea un método registrarEmpleado que reciba nombre y salario, lo guarde en ambos contenedores y no devuelva nada (void).
4. **El Módulo de Consulta:** Crea un método obtenerSalario que reciba el nombre y devuelva el salario (double).

```java
import java.util.*;

public class GestionTalento {
    // Declaramos las estructuras como estáticas para usarlas en los métodos
    static List<String> listaNombres = new ArrayList<>();
    static Map<String, Double> mapaSalarios = new HashMap<>();

    public static void main(String[] args) {
        // Ejecución de la lógica
        registrarEmpleado("Ana", 2500.0);
        registrarEmpleado("Pedro", 1800.0);

        System.out.println("Nombres registrados: " + listaNombres);
        System.out.println("Salario de Ana: $" + obtenerSalario("Ana"));
    }

    // Método modular para registro
    public static void registrarEmpleado(String nombre, double salario) {
        listaNombres.add(nombre);
        mapaSalarios.put(nombre, salario);
        System.out.println("Sistema: " + nombre + " registrado exitosamente.");
    }

    // Método modular para consulta
    public static double obtenerSalario(String nombre) {
        if (mapaSalarios.containsKey(nombre)) {
            return mapaSalarios.get(nombre);
        }
        return 0.0; // Retorno por defecto si no existe
    }
}
```

Refuerzo Independiente - El Gestor de Inventario de Precios

Ahora tú solo crearás un sistema que gestione productos. Debes permitir agregar un producto y luego aplicar un descuento masivo.

Tu guía de pasos:

1. Crea un HashMap<String, Double> llamado productos.
2. Crea un método agregarProducto(String nombre, Double precio).
3. Crea un método llamado aplicarDescuento que reciba el nombre del producto y el porcentaje (ej: 0.10 para el 10%). Este método debe **actualizar** el valor en el mapa.
4. Crea un método mostrarCatalogo que imprima todos los productos usando un ciclo for sobre entrySet().

Instrucción de apoyo: Recuerda que para actualizar en un Map simplemente usas .put() con la misma llave y el nuevo valor calculado.

Ejercicio Independiente Aplicado: "El Sistema de Turnos de un Banco"

Instrucciones de la Actividad: Eres el desarrollador de un banco. Debes crear un programa que:

1. Use un ArrayList para gestionar la fila de espera de clientes (los clientes entran y salen en orden).
2. Use un HashMap para registrar cuántas transacciones ha hecho cada cliente en el día (Llave: nombre, Valor: cantidad de transacciones).
3. **Requerimiento Modular:** Debes crear un método atenderSiguiente() que:
   - Tome al primer cliente de la lista.
   - Le sume 1 a su contador de transacciones en el mapa.
   - Lo elimine de la lista de espera.
   - Muestre un mensaje: "Atendiendo a [Nombre]. Transacciones totales hoy: [Cantidad]".

Criterios de Logro:

- Eliminar correctamente el elemento en el índice 0 del ArrayList.
- Verificar si el cliente ya existe en el Map antes de sumar (si no existe, su primera transacción es 1).
- Uso de métodos con parámetros adecuados.

¿Qué más debería saber?

La trampa del for-each y la eliminación:

Nunca elimines un elemento de un ArrayList mientras lo recorres con un for-each básico; Java lanzará una ConcurrentModificationException. Si necesitas borrar mientras recorres, usa un Iterator o un ciclo for invertido (desde el final hacia el inicio).

Programar para la Interfaz:

Nota que en los ejemplos uso List<String> lista = new ArrayList<>();. Esto es una **buena práctica profesional**. Si mañana decides que ArrayList es lento y quieres LinkedList, solo cambias la parte derecha del =.

CESDE | comfama